

Distribuirani sistemi

Skripta 2022

Pitanja su izvučena sa svih blanketa trenutno dostupnih na arhivi blanketa SICEF

Rešenja blanketa:

- Jun 2020: <https://blanketi.sicef.info/elfak/pregled/2535>

- Procesi P1, P2, P3, i P4 dele isti resurs (kriticnu sekciju - KS). Za postizanje uzajamnog iskljucivanja koristi se Ricart-Agrawala algoritam. Proces P1 se trenutno nalazi u KS. Dok jos traje pristup KS, procesi P4, P2 i P3 (tim redosledom) izdaju svoje zahteve da udju u istu KS. Pokazati sadrzaj redova cekanja u svakom od procesa po pravilima kako funkcioniše algoritam dok **2**

- je P1 jos u KS

Ako se salju po po redosledu P4, P2, P3 znaci da su im recimo vremenske markice 1, 2, 3 respektivno. Nacrtamo sliku i primenimo pravila.

P1: P4, P2, P3 - kada je u KS sve zahteve smesta u red cekanja (ne odgovara sa OK)

P2: P3

P3:

P4: P2, P3

- kada je P1 napustio KS

P1: - kad okonca pristup KS salje potvrdu OK svima iz reda cekanja.

P2: P3

P3:

P4: P2, P3

- U grupi postoji pet repliciranih procesa. Ako mogu nastupiti samo mirne greške, koliko maksimalno procesa može otkazati a da se ipak dobije korektan rezultat? Ako mogu nastupiti greške vizajntijskog tipa, koliko procesa može maksimalno otkazati a da se ipak dobije korektan rezultat. Šta ako mogu nastupiti greške vizantijskog tipa a procesi moraju prostići konsenzus? Koliko u ovom slučaju maksimalno procesa može otkazati? Svaki odgovor obrazložiti.

Mirne greske: 4 jer je $k + 1 = 5$ (4 procesa mogu otkazati jer mora da ostane bar jedan ispravan)

Vizantijske greske: 1, jer je $2k + 1 = 5$ (1 proces moze otkazati jer mora da ostane bar $\frac{2}{3}$ ispravnih)

Da bi se postigao **konsenzus** svi procesi koji ispravno funkcionišu moraju da donesu isti odluku.

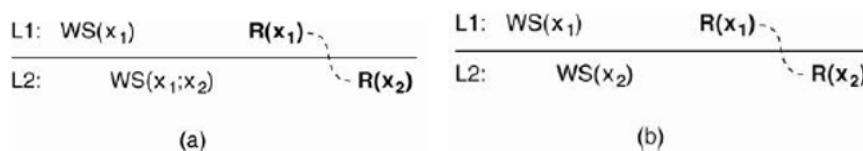
Da bi mogli da tolerisemo m procesa koji ne funkcionišu kako treba neophodno je $2m+1$ procesa koji funkcionišu ispravno, sto znaci da sme da otkaze samo 1 proces.

- Objasniti client centric model konzistencije, varijantu monotona čitanja (monotonic reads). Dati primer skladišta podataka koje je konzistentno i koje nije konzistentno u odnosu na ovaj model konzistencije. Kod **client centric** modela konzistencije naglasak je na održavanju konzistentnog pogleda na skladište podataka sa stanovišta pojedinačnog klijentskog procesa.

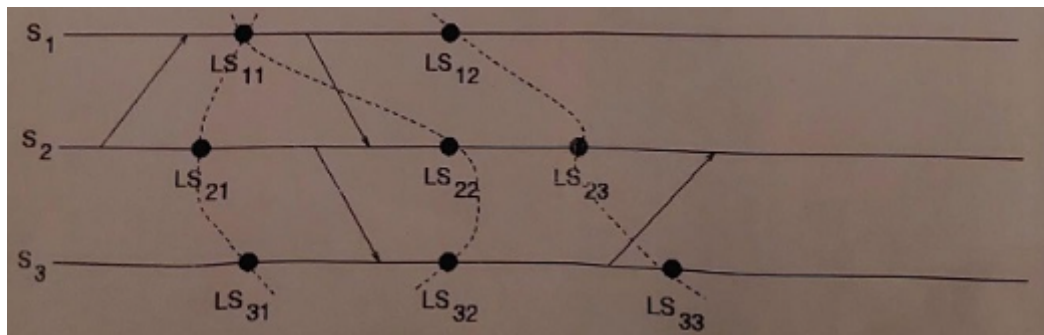
Klijent je mobilan i može sa različitih lokacija čitati-modifikovati kopije skladišta podataka.

Monotonic reads (monotona čitanja): Ako proces pročitao vrednost podatka x na jednoj lokaciji, svako novo čitanje tog podatka će vratiti istu ili noviju vrednost bez obzira na kojoj lokaciji se obavlja čitanje.

- (konzistentno)** Proces P prvo čita x na lokaciji L1 i vidi vrednost x_1 što je rezultat operacije $WS(x_1)$ koja je obavljena na L1. Kasnije P čita x sa lokacije L2, da bi se garantovala konzistentnost skup operacija $WS(X_1)$ mora da bude propagiran i na L2 pre drugog čitanja.
- (nije konzistentno)** Nakon što je proces P pročitao x_1 na lokaciji L1, kasnije čita x_2 sa lokacije L2. Na lokaciji L2 su obavljene samo operacije $WS(X_2)$ nema garancija da ovaj skup $WS(X_2)$ sadrži sve operacije koje su bile obavljene u skupu $WS(X_1)$.



4. Šta je konzistentni presek? Koje linije na slici dole čine a koje ne čine konzistentni presek? Obavezno dati obrazloženje za svaki odgovor.



Svaki proces povremeno pamti svoje stanje na disku (vr. promenljivih, primljene i poslate poruke...) u tackama provere.

Ako nastupi greska proces se vraća u stanje pre nastupanja greske na osnovu podataka zapamcenih u checkpoint.

U distribuiranom sistemu potrebno je da se svi procesi vrate u stanje odakle je moguće izvršiti oporavak, tj. potrebno je pronaći liniju oporavka (**konzistentni presek**).

- Linija(11, 21, 31) - **jest** konzistentni presek jer nema primljenih poruka koje nisu poslate ili poslatih poruka koje nisu primljene, npr s2 salje poruku s1 koju on prima pre preseka.
- Linija(11, 22, 32) - **nije** konzistentni presek jer S2 prima poruku od S1 pri čemu je ta poruka od strane S1 poslata nakon checkpoint-a.
- Linija(12, 23, 33) - **nije** konzistentni presek jer S3 salje poruku pre checkpointa a S2 je prima nakon checkpoint-a.

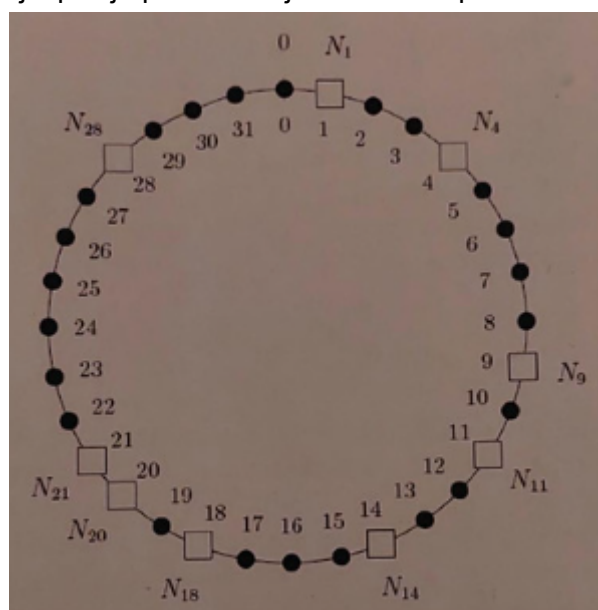
5. Na slici je prikazan Chord P2P.

- a. Koji čvor je odgovoran za ključ 13, a koji za ključ 9?

Za 13 je odgovoran N14

Za 9 je odgovoran N9

- b. Pretpostavimo da se čvor N15 pridružuje Chord prstenu tako što kontaktira čvor N20. Redom napisati korake koji opisuju pridruživanje čvora N15 prstenu.



- N15 pridružuje i kontaktira N20.
- N15 poziva funkciju join(N20) gde će čvor N20 da pronađe neposrednog sledbenika za čvor N15 što je u našem slučaju čvor N18.
- Zatim čvor N15 obavestava čvor N18 da je on njegov sledbenik, čvor N18 će nakon ovoga staviti N15 za svog novog prethodnika.
- Što se tiče prethodnika čvora N15 on će biti dodat tek kasnije kada N14 pokrene protokol za stabilizaciju kojim pita onaj čvor za koj smatra da mu je sledbenik. Ko je njegov prethodnik?
- N14 pita N18 ko je njegov prethodnik i N18 mu odgovara sa N15 čime N14 postavlja N15 za svog sledbenika i obavestava ga o tome

- Zatim N15 postavlja N14 za svog prethodnika

6. Šta je sekunda preskoka i za šta se koristi?

Zbog oscilacija u brzini rotacije zemlje dolazi do razlike između **atomske** vremena i **solarnog** vremena.

BIH (internacionalni biro za vreme) rešava problem usaglašavanja solarnog i atomske vremena tako što kada razlika postane **800ms**, ubacuje se **prestupna sekunda (leap second)** u **TAI** vreme i tada TAI dan traje **864001 sec**.

7. Šta su algoritmi izbora? Navesti neke i objasniti jedan od njih.

Algoritmi izbora su algoritmi za selekciju **koordinatora** ili **servera**. Cilj algoritama izbora je da pronađu aktivni proces sa najvećim identifikatorom i proglase ga za koordinatora.

- **Bully**
- **Ring**

BULLY objašnjenje

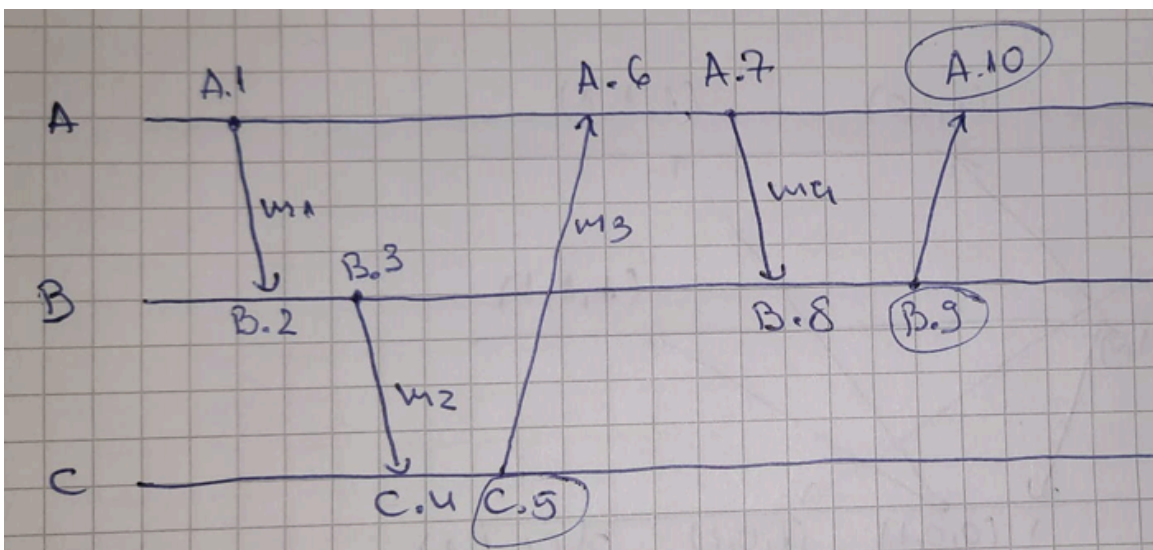
- Proces Pi koji detektuje da koordinatorski ne odgovara startuje izbor.
- Izborna poruka se šalje **svim procesima sa većim identifikatorom** i čeka se odgovor.
- Ako u okviru određenog vremenskog intervala ne stigne odgovor ni od jednog procesa, to znači da ni jedan proces sa **većim** identifikatorom nije aktivan i pobednik je proces koji je inicirao izbor (tj. Pi).
 - Pi obaveštava sve procese da je on koordinatorski.
- Ako pristigne odgovor na poruku izbora, proces Pi se povlači iz izbora i čeka da dobije poruku od novog koordinatora.

8. Objasniti razliku između mrežnog OS, distribuiranog OS i OS baziranog na middleware.

9. Tri računara A, B i C komuniciraju koristeći protokol koji implementira Lamportove vremenske markice.

Na početku u svim računarima vrednosti vremenskih markica su 0. Kasnije nastupaju sledeći događaji:

- A šalje poruku M1 u B: "Zdravo"
 - Nakon što je primio M1, B šalje poruku M2 u C: "A mi se javio"
 - Nakon što je primio M2, šalje poruku M3 u A: "B mi dosađuje"
- Uz svaku od poslanih poruka napisati vrednost vremenske markice sa kojom je poslata:
 - Send(M1, 1)
 - Send(M2, 3)
 - Send(M3, 5)
 - Nakon ovih događaja, poslate su još tri poruke:
 - Nakon što je primio poruku M3, A šalje poruku M4 u B: "C je nedosledan"
 - Nakon što je primio poruku M4, B šalje poruku M5 u A: "C mi dosađuje"
 - A prima poruku M5.



Nakon što su sve poruke poslate i primljene, kolike su vrednosti vremenski markica u svakom od računara A, B i C?

A = 10

B = 9

C = 5

iii. Da li opisani slučaj predstavlja primer relativne ili potpuno uređene komunikacije?

U pitanju je relativno uređenje događaja, tj. nema multicast-a.

10. U distribuiranom sistemu klijent koristi RPC za komunikaciju sa serverom. Klijent šalje zahtev serveru (npr. operation x()) sve dok ne dobije odgovor od servera.

a. Koja RPC semantika u slučaju greške je u ovom slučaju implementirana?

U pitanju je **at-least-one** semantika jer klijent šalje zahtev sve dok ne primi odgovor od servera, odnosno operacija će se izvršiti najmanje jednom a možda i više puta.

b. Objasniti kako mora da bude implementiran server da bi se ovakva semantika obezbedila?

Dve mogućnosti na strani servera:

- Da implementira **idempotentnu** proceduru operation x(). (idempotentna procedura vraća uvek isti rezultat ma koliko se puta izvrši)
- Da server bude **statefull** i da vodi računa o klijentskim zahtevima i prepozna duplikate zahteva i u tom slučaju vrši samo retransmisiju poruke u kojoj je odgovor (ne izvršava ponovo proceduru operation x()).

11. Ako je P2P organizovan kao Chord i ima N čvorova, u koliko maksimalno koraka se može pronaći željeni sadržaj ako postoji?

Ako ima N čvorova, sadržaj se pronalazi u najviše $\log(N)$ koraka.

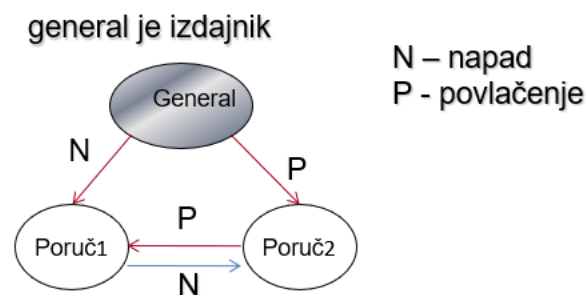
E sad uvek može da se uradi obilazak sukcesivnim kontaktiranjem naslednika i time pronaći sadržaj ako postoji, onda je broj koraka jednak sa brojem čvorova N.

Ne znam na šta je mislila ali bolje oba da se napisu, možda je trik pitanje

12. U Chord sistemu čvor q je sledbenik čvora p. U toku rada sistema čvor p otkriva da njegov pokazitelj na sledbenika više nije validan jer je čvor q ažurirao pokazitelja na svog prethodnika. Šta je uzrok ovoga?

Dodat je novi čvor između p i q.

13. Pomoću primera pokazati da nije moguće postići konsenzus između ispravnih procesa ako postoje 3 procesa od kojih jedan ispoljava grešku vizantijskog tipa.



Potrebno je da za m procesa koji ne daju tačan rezultat imamo $2m + 1$ dobrih procesa.

$m = 1, n = 3$

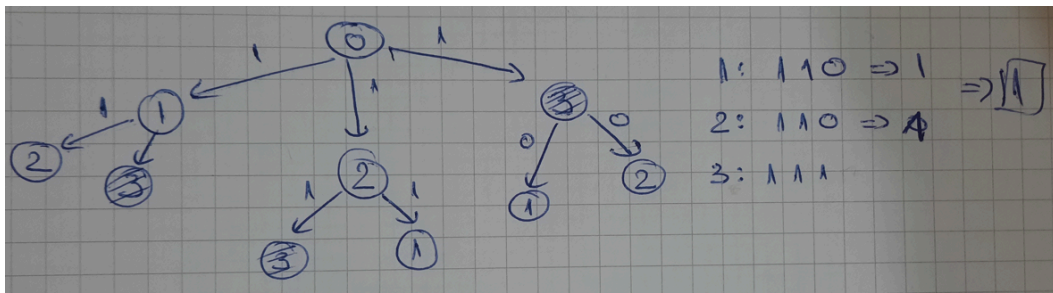
$2m + 1 = 3 \Rightarrow$ **potrebno je 4 procesa da bi se donela odluka sa jednim neispravnim procesom.**

14. Pomoću primera pokazati da je moguće postići konsenzus između ispravnih procesa koji ako prostoje 4 procesa od koji jedan ispoljava grešku vizantijskog tipa.

Napomena: Obavezno navesti koji uslovi moraju biti zadovoljeni kod pristizanja konsenzusa.

Mora se postići da

- Svi lojalni poručnici donesu istu odluku
- ako je general lojalan, tada svaki lojalni poručnik poštuje generalovo naređenje



15. SUN ov NFS koristi montiranje (mount points) da bi dodelio lokalna imena udaljenim fajlovima.

Napomena: Obavezno obrazložiti a i b.

a. Da li je ovaj metod imenovanja fajlova lokaciono transparentan?

Da, kada se ostvari povezivanje sa udaljenim serverom klijent ne mora da vodi racuna da li je fajl kome pristupa lokalni ili udaljeni.

b. Da li je lokaciono nezavistan?

Ne, ne podrzava migracionu transparentnost.

c. Navesti bar jedan nedostatak korišćenja "mount points" u distribuiranom fajl sistemu.

Nedostatak: isti fajl može imati više imena u zavisnosti od klijenta.

16. Kako izgleda poziv Sun RPC kompajlera? Koji fajlovi se dobijaju kao rezultat ovog kompajliranja i šta sadrže? Objasniti na primeru.

Poziv Sun RPC kompajlera:

rpcgen -C primer.x

Dobijaju se sledeci fajlovi:

primer.h (header fajl) - sadrzi jedinstveni identifikator interfejsa, definiciju tipova, konstanti, i prototipova funkcija.

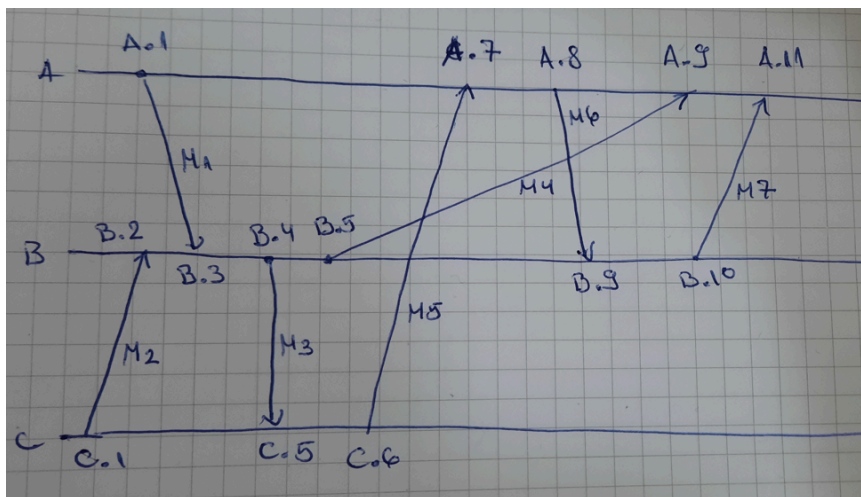
primer_clnt.c (klijent stub) - sadrzi procedure koje ce klijent program pozivati.

primer_svc.c (server stub) - sadrzi procedure koje se pozivaju kada stigne poruka do servera i koje zatim pozivaju odgovarajucu serversku proceduru.

17. Tri računara, A, B i C. Komuniciraju i koriste protokol koji implementira Lamportove logičke časovnike.

Na početku su svi logički časovnici u sva tri računara postavljeni na nulu. kasnije nastupaju sledeći događaji:

- A šalje poruku M1 u B
- C šalje poruku M2 u B
- B prima poruku M2 pre poruke M1
- B odgovara porukom M3 prvo računaru C, a zatim porukom M4 računaru A
- Nakon prijema poruke M3, C šalje poruku M5 računaru A.
- Nakon prijema poruke M5, A šalje poruku M6, računaru B.
- Nakon prijema poruke M6, B šalje poruku M7 računaru A.
- Poslednja poruka koju A prima pre poruke M7 je poruka M4.
 - Označiti kako izgledaju vremenske markice poruka M1 do M7 prilikom slanja
 Send(M1, **A.1**) Send(M2, **C.1**) Send(M3, **B.4**) Send(M4, **B.5**)
 Send(M5, **C.6**) Send(M6, **A.8**) Send(M7, **B.10**)
 - Navesti bar jedan par događaja koji su međusobno uslovljeni, a čija je kauzalnost korektno identifikovana Lamportovim markicama.
A šalje poruku M1 u B, B nakon prijema šalje poruku M3 u C.
 - Da li se događaji kada B šalje poruku M4 i kada C šalje poruku M5 mogu identifikovati kao konkurentni događaji korišćenjem Lamportovih vremenski markica. Objasniti zašto.



18. Časovnik na gradskom tornju u gradu A je bio pokvaren. Časovnik je popravljen, ali sada mora da bude podešen. Voz napušta stanicu u gradu A i odlazi u grad B koji je 200km udaljen. Četiri sata kasnije voz se vraća sa informacijom da vreme po časovniku u gradu B iznosi 6:15. Ako se koristi Kristijanov algoritam na koju vrednost treba da bude postavljen časovnik u gradu A?

$$T_{\text{novo}} = \text{Cutc} + (T_1 - T_0) / 2$$

$$T_0 = 2:15$$

$$T_1 = 6:15$$

$$\text{Cutc} = 6:15$$

$$T_{\text{novo}} = 6:15 + (6:15 - 2:15) / 2 = 6:15 + 4/2 = 8:15$$

19. Nad objektom x obavljene su sledeće operacije u distribuiranom skladištu:

- P1: write(x)A
- P2: write(x)B, read(x)C
- P3: read(x)B, read(x)A, write(x)C
- P4: read(x)B, read(x)A

Nacrtati kako izgleda stvarni redosled događaja ako je skladište podataka sekvencijalno konzistentno.

20. Koje vrste HDFS demona postoje i koja je uloga svakog od njih? Opisati postupak čitanja i postupak upisa u HDFS.

- **NameNode**
 - NN je centralni kontroler HDFS-a
 - NN čuva metapodatke fajl sistema, nadgleda ponašanje DN-a i koordiniše pristup podacima.
 - NN ne čuva podatke nad kojima se vrši obrada, on samo ima informaciju o blokovima koji čine fajl i gde su ti blokovi locirani u klasteru
- **DataNode**
 - DataNode odgovoran za čuvanje podataka u blokovima, primanje naredbi od NN i davanje informacija NN-u.

Upis:

- Klijent je spreman da loaduje fajl File.txt u klaster (razbija ga u blokove A,B i C) počevši od bloka A.
- Klijent kontaktira Name Node, konsultuje ga, dobija dozvolu od NN i dobija listu od (3) DataNoda za svaki blok koji treba da bude upisan (jedinsvenu listu za svaki blok).
- NN koristi svoje Rack Awareness podatke da bi uticao na odluku koji DataNodovi će se naći na listi za pojedinačni blok
- Ključno pravilo je da za svaki blok podataka, jedna kopija se nalazi u jednom reku, a druge dve kopije u reku različitom od ovog s tim što su te dve kopije u istom reku.
- Sve liste koje dobija Klijent moraju da poštuju ovo pravilo
- Za blok A na listi su DN1, DN5 i DN6.
- Pre nego što Klijent upiše Blok A fajla File.txt u klaster, on želi da zna da su svi DataNodovi u koje treba da upiše spremni da prime blok.

- Nakon stizanja potvrde, od DN6 do DN5, od DN5 do DN1, i od DN1 do klijenta, Klijent je spreman da upiše blok A.
- Prilikom upisa DN-ovi formiraju pipeline, u redosledu koji minimizira rastojanje od klijenta do poslednjeg DN-a

Citanje:

- Kad Klijent želi da pročita rezultujući fajl iz HDFS, konsultuje NN i pita za lokacije blokova u rezultujućem fajlu
- NN za svaki blok vraća listu DN-ova koji ga sadrže
- Klijent bira za svaki blok prvi DN iz liste (lokacije blokova su sortirane prema udaljenosti od klijenta) tj. najbližu repliku
- Klijent čita blokove sa odgovarajućih DN-a sekvencijalno, jedan po jedan

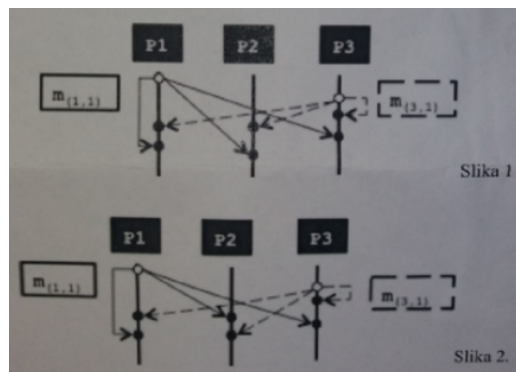
21. Šta predstavlja transakcija u distribuiranim informacionim sistemima? Koje osobine treba da zadovolji svaka transakcija da bi bila uspešno izvršena?

Transakcija predstavlja skup operacija koje se obavljaju kao jedna nedeljiva (atomična) operacija.

Svaka transakcija mora da zadovolji ACID test pre nego što se dozvoli modifikacija sadržaja

- **A – Atomicity**
 - podrazumeva da se transakcija obavi kompletno ili se uopšte ne obavi
- **C - Consistency**
 - transakcija ne ugrožava skup invarijanti (ograničenja, konstanti) sistema.
 - Npr. ako je uslov da sve transakcije nad bazom podataka moraju imati pozitivnu vrednost, bilo koja transakcija sa negativnom vrednošću će biti odbijena. (npr. ako pokušate da podignete više novca nego što imate na računu)
- **I – Isolation**
 - transakcije su međusobno izolovane, tj. dve konkurentne transakcije moraju biti serijalizovane (izvršavaju se u nekom nedeterminisanom redosledu, ali taj redosled mora biti vidljiv na isti način za sve transakcije)
- **D - Durability**
 - kada se transakcija obavi ne može se poništiti!

22. Šta se podrazumeva pod pojmom potpuno uređena grupna komunikacija (total ordering). Na slikama 1 i 2 prikazano je izvršenje (slanje i prijem poruka) u tri procesa P1, P2 i P3. Poslate su dve poruke m(1,1) i m(3,1). Za svaku sliku odgovoriti da li predstavlja “total ordering” ili ne. Obrazložiti odgovore.



Pod pojmom potpuno uređena grupna komunikacija se podrazumeva

- Potrebno je obezbediti da se operacije azuriranja obave na obe strane u istom redosledu.
- Neophodno je obezbediti potpuno uređenu grupnu komunikaciju (multicast) tj. operaciju kojom se sve poruke isporučuju u istom redosledu svim prijemnicima.

23. Koje se tehnike mogu koristiti za postizanje visoke pouzdanosti sistema? Objasniti erasure coding tehniku.

Opšti prilaz u projektovanju sistema otpornih na greske je redudansa.

- **TMR (Triple Modular Redundancy)** - tehnika za postizanje visoke pouzdanosti kroz fizičku redundantnost.
- **ABFT tehnika (Algorithm base fault tolerance)** - informaciona redudansa.
- **Erasure coding** - tehnika za postizanje otpornosti na greške (fault tolerance)

Erasure coding

Tehnika kodiranja kod koje se skup od k podataka kodira skupom od n podataka pri čemu je $n > k$ tako da se na osnovu bilo kog skupa od k podataka mogu regenerisati originalni podaci.

(n, k) erasure code omogućava **tolerisanje $n - k$ grešaka**.

Tehnika koja je široko prihvaćena kod HDF (Hadoop file system)

- Posmatrajmo fajl veličine M
- Podeliti fajl na k blokova veličine M / k
- Primeniti (n, k) code na ovih k blokova da bi se dobilo n blokova, svaki veličine M/k .
 - Na taj način fajl je proširen n/k puta
 - Potrebno je da n bude veće ili jednako k .
 - Ako je n jednako k , fajl je samo podeljen na blokove, ali nema nikakvog kodiranja
- Bilo koji podskup k od n blokova se može uzeti da bi se rekonstruisao fajl
 - to znači da kod može tolerisati $(n-k)$ grešaka

24. Koje greške mogu nastupiti u klijent server komunikaciji kod RPC i koje se akcije preduzimaju kada nastupe odgovarajuće greške.

- **klijent nije u stanju da locira server** - kad se desi ovakva greska neophodno je da se emituje exception.
- **zahtev upućen od klijenta ka serveru je izgubljen** - klijent treba da startuje casovnik kada uputi zahtev. Ako istekne timeout pre nego sto stigne odgovor ili potvrda vrsi se retransmisija.
- **otkaz servera nakon prijema zahteva od klijenta** -Dva rešenja su moguća:
 - (1) Klijent šalje zahteve sve dok ne dobije odgovor (**at least one** semantika – garantuje da će se RPC izvršiti bar jednom, a možda i više puta - primenljivo kod idempotentnih funkcija)
 - (2) Odmah signalizirati grešku nakon isteka time-outa (**at most one** semantika – RPC će se izvršiti najviše jednom, a možda i ni jednom)
- **odgovor servera klijentu izgubljen** - klijent ponovo salje zahtev s tim da se zahtevima dodeljuju redni brojevi. Server vodi racuna o rednim brojevima klijentskih zahteva moze otkriti duplikat i odbiti da izvrši zahtev ponovo.

25. Kada smo govorili od DFS rekli smo da server može biti projektovan kao statefull ili stateless. Pored svakog od sledećih tvrdnji staviti ozanku tačno T ili netačno F.

- a. Zaključavanje fajla je teško implementirati kod stateless servera. **nzm, pise da je kod statefull moguće implementirati zakljucavanje, tkd ovde vrv ide T**
- b. Lakše je izboriti se sa greškama kod statefull nego kod stateless servera. **F**
- c. Implementacija klijentske strane može biti komplikovanija sa statefull serverom **T**
- d. Kod stateless servera, svaki klijentski zahtev mora da sadrži kompletnu informaciju o zahtevu(npr. ime fajla, offset, itd.) **T**

26. Šta omogućava uspešan restart servera na kom se izvršava NameNode demon? Koji se nepohodni podaci za funkcionisanje klastera a koje poseduje NameNode, i gde i kako se oni skladište pre i nakon restarta severa.

Checkpoint i edit log. Pri restartu NN promene iz edit loga se primenjuju na poslednju verziju Namespace fajla (check point) da bi se dobila nova verzija metapodataka fajl sistema.

Podaci neophodni za funkcionisanje klastera su **File namespace i File to block mapping**.

Pre restarta NameNode podatke koji su u glavnoj memoriji prebacuje u perzistentno skladište, kako bi se sacuvala njihova vrednost. Nakon restarta podaci se ponovo upisuju u glavnu memoriju.

27. Koje su prednosti distribuiranih sistema u odnosu na centralizovane?

- **Ekonomski** - DS imaju bolji odnos cena/performance od velikih (mainframe) računara.
- **Brzina** - DS može imati veću ukupnu moć obrade nego mainframe.
- **Pouzdanost** - Ako imate centralizovani sistem i računar otkáže, aplikacija ne može da se izvršava. Kod DS (ako je dobro projektovan) ako 15% mašina otkáže, aplikacija i dalje može da se izvršava, tj. sistem i dalje može ostati u funkciji sa samo 15% degradacije u performansama.
- **Deljenje resursa** - Omogućeno je da više korisnika pristupa zajedničkim bazama podataka i periferijama.

- Tipčan primer distribuirani fajl sistem
- web
- **Komunikacija** - olakšana komunikacija između ljudi.
- **Efikasno korišćenje resursa** - omogućava da se opterećenje rasporedi na raspoložive računare na najefikasniji način

28. Zašto je teško ostvariti sinhronizaciju u DS?

Nepostojanje globalnog časovnika.

29. Zašto je (nekada) problem detektovati greške u DS?

Verovatno zato što kod gresaka vizantijskog tipa komponenta radi, idalje generise rezultate, samo netacne. Nema jake naznake da komponenta ne radi. Dok kod mirnih gresaka komponenta nije u funkciji uopste.

30. Pre nego što klijent uputi RPC poziv serveru, server mora da bude registrovan. Kako se obavlja registrovanje servera? Šta je portmapper? Šta se podrazumeva pod terminom "binding" (povezivanje)? Server se **registruje** u direktorijumskom servisu tako što dostavlja adresu serverske masine, ime servera i interfejse koje implementira.

Portmapper je poseban program na udaljenom hostu koji pamti preslikavanja imena programa i broja verzije u broj porta.

Povezivanje (binding) - da bi se realizovao poziv udaljene procedure neophodno je locirati udaljeni host i odgovarajući proces na hostu.

31. Navesti pravila za implementaciju vektorskih časovnika. Da li se na osnovu vektorskih časovnika može utvrditi da li su dva događaja međusobno uslovljena? Obrazložiti odgovor.

Pravila:

1. Vektor se inicijalizuje na 0 u svim procesima: $V[i,j]=0$, za $i,j=1,\dots,N$
2. Proces P_i inkrementira i -ti element vektora u lokalnom vektoru pre nego što obeleži događaj: $V[i,i]=V[i,i]+1$. Poruka se šalje iz procesa P_i zajedno sa V_i .
3. Kada P_j primi poruku poredi lokalni vektor sa primljenim, element po element, i postavlja element u lokalnom vektoru na veću od vrednosti

Na osnovu vektorskih časovnika može se utvrditi da li su dva događaja međusobno uslovljena.

Obrazloženje:

Za bilo koja dva događaja e i e' važi sledeće

ako je $e \rightarrow e'$ tada $V(e) < V(e')$

ako je $V(e) < V(e')$ tada je $e \rightarrow e'$

Ako se ne može uspostaviti relacija $V(e) \leq V(e')$ ili $V(e) \geq V(e')$, tada su događaji konkurentni (tj. nisu međusobno uslovljeni)

32. U procesu množenja dve matrice korišćena je ABFT tehnika za detekciju i korekciju grešaka. Dobijen je sledeći rezultat:

1	2	3	6
2	3	4	9
3	3	3	8
6	7	10	23

- a. Utvrditi da li je nastupila greška u toku izračunavanja
Sumom odgovarajućih kolona i vrsta utvrđeno je da ima neslaganja, samim tim i greske.
- b. Ako je nastupila greška, naći njenu lokaciju
Presek treće vrste i druge kolone
- c. Izvršiti korekciju greške, ako je greška nastupila
 $C_{32} = 3 + (7 - 8) = 3 - 1 = 2$
ili po vrsti
 $C_{32} = 3 + (8 - 9) = 3 - 1 = 2$

33. Objasniti uz pomoć primera šta se podrazumeva pod pristupnom, lokacionom i migracionom transparentnošću.

- **Pristupna transparentnost** - podacima i resursima se pristupa na jedinstven način, bez obzira da li se oni nalaze na udaljenom ili lokalnom računaru
 - (Primer) Različiti OS mogu koristiti različite načine imenovanja fajlova. Razlike u imenovanju fajlova i način manipulacije fajlovima mora biti skriven za korisnika i aplikaciju
- **Lokacijska transparentnost** - korisnik ne zna (i ne mora da zna) gde je resurs fizički lociran u sistemu.
 - (Primer) URL imena: `http://www.elfak.com/index.htm` ne nagoveštava gde se nalazi glavni web server elektronskog fakulteta.
 - Korisnik sa bilo koje lokacije na isti način pristupa resursu
- **Migraciona transparentnost** - resurs može promeniti lokaciju a da klijent to ne primeti. Resurs se može kretati a da pri tome ne dolazi do promene imena resursa.

34. U distribuiranom sistemu tri procesa P0, P1, P2 međusobno komuniciraju. U procesu P0 vektorski časovnik ima vrednost (8, 2, 4). Koja poruka iz procesa P2 može odmah biti prosleđena aplikaciji u procesu P0 ako je neophodno očuvati uređenje međusobno zavisinih događaja **2**

- (8, 2, 6)
- (9, 3, 5)
- (1, 1, 5)**
- (4, 1, 3)

Obrazložiti kako ste došli do odgovora.

a) primljena poruka: `tmp = (8, 2, 6)`

lokalni vektor u P0 je: `V = (8, 2, 4)`

Proveravamo uslove:

A) `tmp[3] = 6`

`V[3] = 4 + 1 = 5, 6 > 5 => NE ISPUNJAVA USLOV A`

B) `tmp[2] = 2 = V[2] = 2; && tmp[1] = 8 = V[1] = 8 => ISPUNJAVA USLOV B.`

c) primljena poruka: `tmp = (1, 1, 5)`

lokalni vektor u P0 je: `V = (8, 2, 4)`

Proveravamo uslove:

A) `tmp[3] = 5`

`V[3] = 4 + 1 = 5, 5 = 5 => ISPUNJAVA USLOV A`

B) `tmp[2] = 1 <= V[2] = 2; && tmp[1] = 1 <= V[1] = 8 => ISPUNJAVA USLOV B.`

35. Definisati pojmove defekt (fault), greška (error) i otkaz (failure) i navesti primer koji ilustruje ova tri pojma.

- **Defekt (fault)** - je trajni ili privremeni nedostatak koji se javlja u nekoj hardverskoj ili softverskoj komponenti.
- **Greska (error)** - greska je posledica defekta, defekt je uzrok greske. Greske su manifestacija defekta i predstavljaju odstupanje vrednosti podatka od očekivane vrednosti.
- **Otkaz (failure)** - otkaz nastupa kada sistem više ne može da obavlja funkciju za koju je projektovan.

Obuhvatni primer:

U softverskom sistemu nekorektna instrukcija u programu koja dekrementira vrednost neke promenljive umesto da je inkrementira je defekt. Kada se ova instrukcija izvrši ona će generisati pogresnu vrednost promenljive. Ako druge naredbe u programu koriste tu vrednost ceo sistem će odstupiti od željenog ponašanja.

U ovom primeru nekorektna instrukcija je **defekt**, Vrednost promenljive je **greska**, a **otkaz** je ponašanje sistema koja je posledica greske.

36. Distribuirani sistem se sastoji od 4 replicirana servera. Pouzdanost svakog pojedinačnog servera je 0.9.

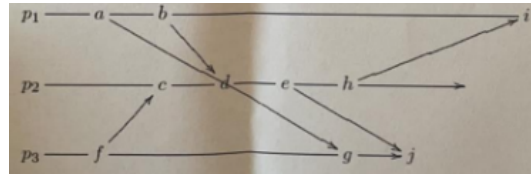
- Ako je sistem projektovan tako da može da funkcioniše ako je bilo koji od servera u funkciji, kolika je pouzdanost celog sistema?

$$P = 1 - (1 - 0.9)^4 = 1 - (0.1)^4 = 1 - 0.1 = 0.9999$$

- b. Ako je sistem projektovan tako da može da funkcioniše samo ako su sva četiri servera u funkciji (ispravna), kolika je pouzdanost celog sistema?

$$P = (0.9)^4 = 0.6561$$

37. Na slici su prikaza tri procesa P1, P2, i P3 koji međusobno komuniciraju. Označiti svaki događaj Lamportovim vremenskim markicama i vektorskim markicama. Da li je moguće da dva događaja imaju jednake vrednosti Lamportovih markica? Ako je moguće, dati primer. Ako nije odgovor.



Moguće je ako se ne koristi prisila da svaka markica bude jedinstvena, odnosno da se pored vrednosti lokalne vremenske markice doda i id odnosno broj procesa koji je globalno jedinstven. **-VALJDA**

38. Definirati kauzalnu konzistenciju. Da li je sledeće skladište podataka kauzalno konzistentno? Obavezno obrazložiti odgovor.

A	W(x)a		W(x)b	
B		R(x)a		W(x)c
C				R(x)b R(x)a

Deff. Upisi koji su potencijalno uslovljeni moraju se videti u svim procesima u istom redosledu. Konkurentni upisi se mogu videti u razlicitim procesima u razlicitom redosledu.

Navedeno skladište jeste kauzalno konzistentno iz razloga sto jedini upis koji je potencijalno uslovljen je upis W(x)c ali on nije procitan u nijednom od procesa s tim u vezi nije narušena konzistentnost.

Drugo, upisi W(x)a i W(x)b nisu uslovljeni tako da mogu da se vide u razlicitim procesima u razlicitom redosledu.

39. Četiri procesa A, B, C, D izvršavaju sledeće aktivnosti nad promenljivim x i y u repliciranom skladištu podataka. da li je prikazano skladište podataka sekvencijalno konzistentno? Da li je skladište podataka kauzalno konzistentno? Obrazložiti.

A	W(x)10		R(y)5	
B		W(x)20		
C	R(x)20		R(x)10	W(y)5
D		R(x)20	R(x)10	R(y)5

Sa stanovišta sekvencijalne konzistencije rezultat je korektan jer svi procesi vide preplitanja operacija na isti nacin.

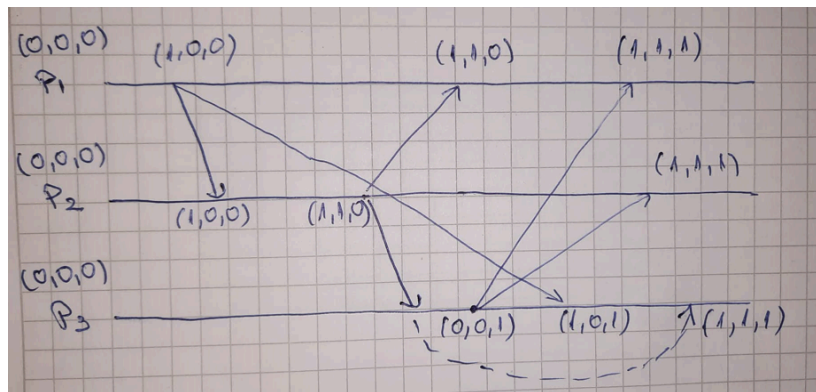
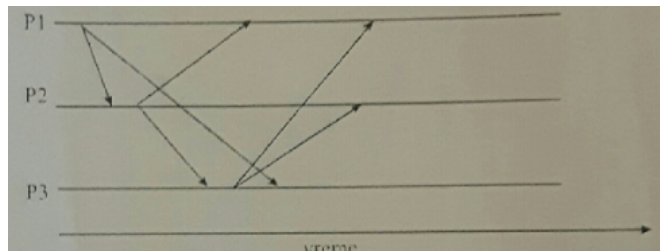
Takođe, sa stanovišta kauzalne konzistencije rezultat je korektan jer nema potencijalno uslovljenih upisa tako da redosled izvršenja nije bitan. Takođe ako je rezultat korektan sa stanovišta sekvencijalne onda mora biti korektan sa stanovišta kauzalne konzistencije jer je kauzalna konzistencija slabiji model konzistencije.

40. Šta je IDL? Koja je njegova uloga u DS?

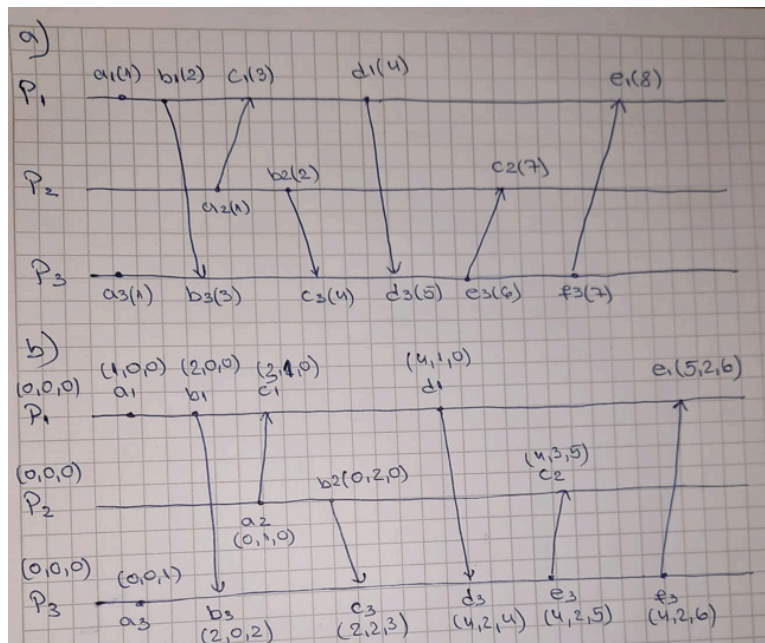
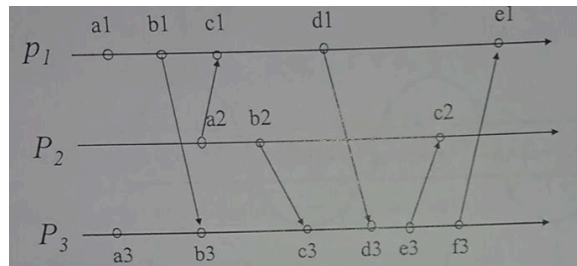
IDL je jezik za definisanje interfejsa.

Opis interfejsa sadrži definiciju funkcija koje su dostupne zajedno sa tipom rezultata, tipom parametra i moguće izuzetke.

41. Na slici prikazana su tri procesa koji međusobno komuniciraju. U svakom procesu logički vektorski casovnici su inicijalizovani na nulu. Pokazati kako se uz pomoc vektorskih casovnika obavlja sinhronizacija međusobno zavisnih događaja. Prikazati vrednosti vektorskih casovnika kod slanja i prijema svake poruke. Takodje, oznaciti poruke koje su baferovane prilikom prijema i trenutak kada se prosledjuju aplikaciji.



42. Na slici prikazana su tri procesa koja medjusobno komuniciraju. Za svaki od događaja dati vrednost
- Lamportovih vremenskih markica
 - Vektorskih časovnika



43. Šta je tačno u Lamportovom sistemu logičkih časovnika?

- if $a \rightarrow b$, then $C(a) < C(b)$
- if $C(a) < C(b)$, then $a \rightarrow b$

44. Da li se na osnovu Lamportovih vremenskih markica može odrediti da li su događaji a i b međusobno uslovljeni? Obrazložiti.

Ne može, zato što kod Lamporta važi da ako je $a \rightarrow b$ onda je $T(a) < T(b)$, ali na osnovu vremenskih markica ne možemo da tvrdimo obrnuto tj. ako je $T(e) < T(b)$ ne znači da je $e \rightarrow b$.

45. Na slici prikazana su tri konkurentna procesa. Inicijalne vrednosti promenljivih x, y i z su 0. Ako proces P1 generise izlaz 00, proces P2 izlaz 11, proces P3 izlaz 10, da li je dobijeni rezultat korektan sa stanovišta sekvencijalne konzistencije? Da li je dobijeni rezultat korektan sa stanovišta kauzalne konzistencije?. Obavezno obrazložiti.

Process P1	Process P2	Process P3
x = 1; print (y, z);	y = 1; print (x, z);	z = 1; print (x, y);

```

x=1
print(y, z)
-----> 00
z=1
printf(x, y)
-----> 10
y=1
print(x, z)
-----> 11

```

Sa stanovišta sekvencijalne konzistencije rezultat je korektan jer ovim sledom izvršenja nije narušen redosled izvršenja definisan u svakom od procesa.

Takođe, sa stanovišta kauzalne konzistencije rezultat je korektan jer nema potencijalno uslovljenih upisa tako da redosled izvršenja nije bitan. Takođe ako je rezultat korektan sa stanovišta sekvencijalne onda mora biti korektan sa stanovišta kauzalne konzistencije jer je kauzalna konzistencija slabiji model konzistencije.

46. Četiri procesa P1, P2, P3 i P4 pristupaju deljivoj promenljivoj x koja se nalazi u lokalnom skladištu podataka. Inicijalno je vrednost promenljive x=0 u svim skladištima. Proces i izvršavaju sledeće aktivnosti

P1	P2	P3	P4
x=2; x=3;	x=1;	while(x= =0); y=x; y=4*y+x	while(x= =0); z=x; z=4*z+x;

- a. Ako je skladište sekvencijalno konzistentno da li je moguće da nakon što se svi procesi izvrše bude y=6 i z=13? Obrazložiti odgovor.

Jedini način da se dobije y = 6 je $y = 4 * 1 + 2$ sto znači da je $y = x = 1$ a to je moguće samo ako se izvrši sekvenca x = 1 pa nakon toga (ne odmah nakon te naredbe) x = 2
Da bi dobili z = 13 $z = 4 * 3 + 1$ sto znači prvo mora da se desi x = 3 pa x = 1

Jedino sto bi dalo ovaj rezultat a da svi procesi vide isti redosled izvršenja je sekvenca x = 3, x = 2, x = 1 sto nije validno jer x = 2 mora da se izvrši pre x = 3 po definiciji sekvencijalne konzistencije mora se ispostaviti redosled operacija pojedinačnih procesa.

- b. Ako je skladište podataka kauzalno konzistentno da li je moguće da nakon što se svi procesi izvrše bude y=6 i z=13? Obrazložiti odgovor.

Moguća je zato što procesi ne moraju da vide isti redosled operacija jer nema uslovljenih upisa.

47. Da li je sledeca tvrdnja tacna? Protokol baziran na postojanju primarne kopije (i backup kopija) je otporniji na otkaze od protokola baziranih na kvorumu:

- tacno**
- netacno

Obavezno obrazložiti odgovor.

Primarni je otporniji na greske jer ako otkaze primar samo se odabere novi bez obzira da li su azuriranja izvršena ili ne. Ako primar otkaze backup ce nastaviti da ispunjava njegovu ulogu. Cak i ako

on otkaze, read operacije u ovom slučaju se obavljaju lokalno (tako da on i dalje čita najsvježije podatke koje postoje)

Kod kvoruma ako neki server otkaze prilikom read ili write operacija može doći do toga da se u sistemu pojave nevalidni podaci.

48. Sta se podrazumeva pod skalabilnim distribuiranim sistemom? Objasniti tehnike za postizanje skalabilnosti.

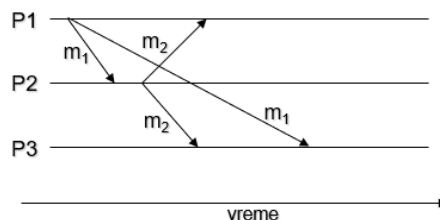
Skalabilnost se može posmatrati kroz tri dimenzije:

- Skalabilnost u odnosu na broj korisnika i resursa
- Skalabilnost u odnosu na geografsku udaljenost resursa i korisnika
- Administrativna skalabilnost – sistemom se može lako upravljati čak i ako se prostire kroz više administrativnih domena.

Tri tehnike:

- **Skrivanje komunikacionog kašnjenja**
 - Koriste se asinhrona komunikacija umesto sinhronih
 - Kada se uputi zahtev udaljenom servisu, klijent se ne blokira dok čeka da stigne odgovor, već radi neki drugi posao
 - Kada stigne odgovor od servera, generiše se prekid koji obaveštava da je poruka stigla
- **Distribucija**
 - Podrazumeva da se komponente dele na manje delove, a zatim se ti delovi distribuiraju na više mašina u sistemu
- **Replikacija**
 - Pomaže da se poveća dostupnost i balansira opterećenje u sistemu da bi se postigle bolje performanse

49. Objasniti kako se uz pomoć vektorskih časovnika može ostvariti uređenje međusobno zavisnih događaja.



Poruka m1 se šalje svim procesima u grupi (multicast)

- Poruka m2 je odgovor na poruku m1 i takođe se šalje svim procesima.
 - Potrebno je osigurati da svi procesi prvo procesiraju poruku m1 a zatim m2.
 - Ako m2 stigne pre mora se baferovati dok ne stigne poruka m1
 - Časovnik se inkrementira samo kod slanja poruke
- + može pravila da se dopisu, nzm dal je potrebno

50. Server je repliciran na 20 čvorova. Za održavanje konzistencije koristi se protokol baziran na kvorumu.

a. Ako se čitanje obavlja samo sa jedne kopije, koliko kopija se najmanje mora azurirati u slučaju modifikacije da bi se obezbedila korektnost?

$N = 20$

$NR = 1$

$NW = ?$

- $NR + NW > N \Rightarrow 1 + NW > 20 \Rightarrow NW = 20$
- $NW > N / 2, 20 > 10$

b. Ako se prilikom upisa azurira 11 kopija, koliko najmanje kopija se mora pročitati da bi se obezbedila korektnost?

$N = 20$

$NW = 11$

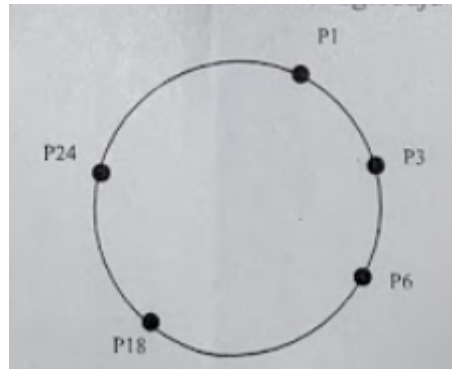
$NR = ?$

- $NR + NW > N \Rightarrow NR + 11 > 20 \Rightarrow NR = 10$

- $NW > N / 2, 11 > 10$

Obrazložiti odgovore.

51. Na slici je prikazan Chord prsten modula 32.



a. Prikazati kako izgledaju Finger tabele svakog cvora.

$2^5 = 32 \Rightarrow$ redovi u finger tabeli idu od 2^0 do $2^{(5-1)} = 2^4$

$P1 + 1 = P3$	$P3 + 1 = P6$	$P6 + 1 = P18$	$P18 + 1 = P24$	$P24 + 1 = P1$
$P1 + 2 = P3$	$P3 + 2 = P6$	$P6 + 2 = P18$	$P18 + 2 = P24$	$P24 + 2 = P1$
$P1 + 4 = P6$	$P3 + 4 = P18$	$P6 + 4 = P18$	$P18 + 4 = P24$	$P24 + 4 = P1$
$P1 + 8 = P18$	$P3 + 8 = P18$	$P6 + 8 = P18$	$P18 + 8 = P1$	$P24 + 8 = P1$
$P1 + 16 = P18$	$P3 + 16 = P24$	$P6 + 16 = P24$	$P18 + 16 = P3$	$P24 + 16 = P18$

b. U kom redosledu ce se posećivati cvorovi pocev od cvora P3 da bi se pronasao sadržaj sa ključem 27?

$P3 \rightarrow P24 \rightarrow P1$

52. Koja je uloga NameNoda kod HDFS? Kako se postize otpornost na greske (fault tolerance) kod HDFS?

Uloga

- NameNode je centralni kontroler HDFS-a
- NN čuva metapodatke fajl sistema, nadgleda ponašanje DN-a i koordiniše pristup podacima.
- NN ne čuva podatke nad kojima se vrši obrada, on samo ima informaciju o blokovima koji čine fajl i gde su ti blokovi locirani u klasteru.
- Takođe vodi računa da svaki blok zadovolji definisani faktor replikacije
- Bez njega klijent ne bi mogao da upisuje i čita fajlove iz
- HDFS-a i bilo bi nemoguće upravljati i izvršavati MR poslove

Otpornost na greske (fault tolerance) - postize se tako sto se blokovi repliciraju na vise cvorova.

Tacnije koristi se tehnika Rack Awareness kojom NameNode osigurava da se za svaki blok podataka jedna kopija upise u jedan rack a preostale dve u drugi rack koji je razlicit od rack-a u kom je upisana prva kopija. Time se postize sledece: da ukoliko crkne ceo rack ne dodje do gubitka svih kopija, ovim su kopije distribuirane po rack-ovima.

53. Pobrojati redom korake kojima se ostvaruje povezivanje klijenta i servera kod DCE.

Da bi klijent mogao da poziva server, neophodno je da server bude registrovan i spreman da primi poziv.

- Registrovanje broja porta servera (processa) u DCE daemonu
- Registrovanje servera u direktorijumskom servisu (adresa serverske mašine, ime servera i interfeisi koje implementira)
- Klijent kontaktira direktorijumski server da dobije adresu server mašine (IP adresu) na kojoj se izvršava server
- Klijent kontaktira DCE daemon da bi dobio broj porta servera kome želi da pristupi
- Poziv udaljene procedure

54. Koje tipove gresaka moze tolerisati sistem koji koristi TMR (trostruku hardversku redudansu)?

Obrazložiti odgovor.

Može tolerisati mirne i vizantijske greske ali samo jednostruke.

Ako dodje do greske na jednoj od komponenti a ostale funkcionisu voteri ce uspesno izglasati rezultat. Npr. ako su 3 ili 2 ulaza u voter ista, izlaz je jednak ulazu. Ako su tri ulaza razlicita, izlaz je nedefinisan, odnosno ukoliko nastupi greska na vise od jedne komponente izlaz je nemoguće odrediti

55. Konzistencija

- a. Sta je striktna konzistencija i zasto je nije moguće postici u distribuiranom sistemu?

Najjači model konzistencije. Bilo koja operacija nad podatkom x vraća rezultat poslednje write operacije nad x. Problem striktna konzistencije je da se bazira na apsolutnom globalnom vremenu i brzina poruke za ažuriranje bi morala da bude 10 puta brža od brzine svetlosti.

- b. Definirati sekvencijalnu i kauzalnu konzistenciju. Dati jedan primer koji ilustruje razlike.

Sekvencijalna konzistencija je slabiji model od striktna. Rezultat bilo kog izvršenja je isti kao da su (read i write) operacije svih procesa na skladištu podataka izvršene u nekom sekvencijalnom redosledu i operacije svakog pojedinačnog procesa pojavljuju se u ovoj sekvenci u redosledu koji je određen njihovim programom.

Kauzalna konzistencija je slabiji model od sekvencijalne. Upisi koji su potencijalno uslovljeni moraju se videti u svim procesima u istom redosledu. Konkurentni upisi se mogu videti u različitom redosledu u različitim procesima.

Primer

P1 w(x)a

P2 w(x)b

P3 r(x)b r(x)a

P4 r(x)a r(x)b

Kod sekvencijalne ovo nije moguće jer svi procesi moraju da vide isti redosled izvršenja, dok kod kauzalne jeste jer konkurentni upisi mogu se videti u različitom redosledu u različitim procesima.

- c. Da li je sledeće skladište kauzalno konzistentno? Obrazložiti odgovor.

A	W(x)a		W(x)b	
B		R(x)a		W(x)c
C				R(x)b R(x)a

Skladište jeste kauzalno konzistentno jer su upisi W(x)a i W(x)c potencijalno uslovljeni i oni se moraju videti u istom redosledu, dok su ostali upisi konkurentni i mogu se videti u različitom redosledu.

56. Objasniti kako se ostvaruje pridruživanje cvora u Gnutella mrezi.

- **Peer čvor X mora prvo pronaći neki drugi peer čvor koji se već nalazi u Gnutella mreži.**
 - koristi listu potencijalnih kandidata za koje se zna da su često prisutni ili kontaktira Gnutella sajt koji sadrži takvu listu
- **X sekvencijalno pokušava da uspostavi TCP konekciju sa peer čvorovima iz liste, dok sa nekim ne uspostavi konekciju, npr. Y**
- **Nakon uspostavljanja TCP konekcije X šalje Ping poruku Y.**
 - Ping poruka sadrži IP adresu i broj porta izvora
 - Y prosleđuje Ping poruku drugim peer čvorovima u overlay mreži
 - Poruka sadrži i brojač koji se dekrementira pri prolasku kroz svaki peer čvor da bi se bujica držala pod kontrolom.
- **Svi peer čvorovi koji prime Ping poruku odgovaraju sa Pong porukom**
 - Pong poruka sadrži IP adresu peer čvora, broj fajlova koje čvor ima na raspolaganju i ukupnu veličinu fajlova u Kbyte
- **X može primiti mnogo Pong poruka.**

57. Objasniti sledeće semantike kod poziva udaljenih procedura:

- a. **mozda (maybe)** - Nema nikakve garancije da će operacija biti izvršena. Klijent ne zna da li je operacija izvršena.

- b. At-least-once - Funkcija će se izvršiti barem jednom, a može i više puta, ako je reč o idempotentnim funkcijama.
- c. At-most-once - Funkcija nije idempotentna i izvršiće se najviše jednom.

58. Vektorski časovnici

- a. Navesti pravila za implementaciju vektorskih časovnika.
 1. Vektori se inicijalizuju na 0 u svim procesima.
 2. Proces P_i inkrementira i -ti element vektora u lokalnom vektoru pre nego sto obeleži događaj.
 3. Kada P_j primi poruku poredi lokalni vektor sa primljenim, element po element, i postavlja element u lokalnom vektoru na veću od vrednosti.
- b. Na osnovu vrednosti vektorskih časovnika koji događaj je prethodio događaju čiji je vektorski časovnik (4, 2, 8, 5)? Dati kratko obrazloženje
 - a. (3, 1, 7, 7)
 - b. (5, 1, 6, 2)
 - c. (4, 2, 8, 4)
 - d. (4, 3, 8, 5)

Objašnjenje

Kod vektorskih časovnika važi da ako je $V(e) \leq V(e')$ tada se e desilo pre e' sto znači događaj koj prethodi je onaj sa manjim vektorskim časovnikom.

A c je jedini manji svi ostali su konkurentni ili veći.

59. Koji tipovi gresaka u odnosu na trajanje postoje? Dati primer za svaku od njih. Objasniti razliku izmedju mirnih i Vizantijskih gresaka. Koju vrstu gresaka je teze detektovati. Obrazloziti.

U odnosu na trajanje

- **Prolazne (transient faults)**
 - Pojave se jednom i nestanu
 - Npr. Istekao time out za prenos poruke, ali nakon ponovnog slanja poruka je uspešno otposlata
- **Peridične (intermittent)**
 - Greška se pojavi, zatim nestane, pa ponovo pojavi. Najnezgodniji tip grešaka
 - Npr greška u mrežnom kablju kod koga postoji neki prekid
- **Stalne (permanent)**
 - Greške koje postoje sve dok se komponenta ne zameni ispravnom
 - Greška u hard disku, pregorela komponenta, bagoviti sw.

Mirna (fail-silent, ili fail-stop) greška - komponenta prestaje sa radom i ne generiše nikakav izlaz ili generiše poruku o grešci.

Vizantijske greške - komponenta nastavlja sa radom i generiše pogrešan rezultat.

Teze je detektovati gresku vizantijskog tipa jer u tom slucaju komponenta funkcionise na neki nacin, tj. generise neki rezultat koji je nekad ocekivani a nekad neocekivani zeljeni rezultat, s tim u vezi je tesko identifikovati tu komponentu.

Lakse je identifikovati komponentu u kojoj nastupa mirna greska jer ta komponenta ne radi, tj. ne generise rezultat.

60. Tri procesa P_1 , P_2 , P_3 pristupaju zajednickim promenljivima x , y i z . Svaki proces ima sopstvenu kopiju skladista podataka u kojoj se nalaze ove promenljive. Inicijalno sve promenljive imaju vrednost 0. U procesima P_1 , P_2 , i P_3 se izvrsavaju sledece naredbe:

P_1	P_2	P_3
$A = 1$		
$x = A$	$y = x$	print y
	$z = y$	print x
		print z

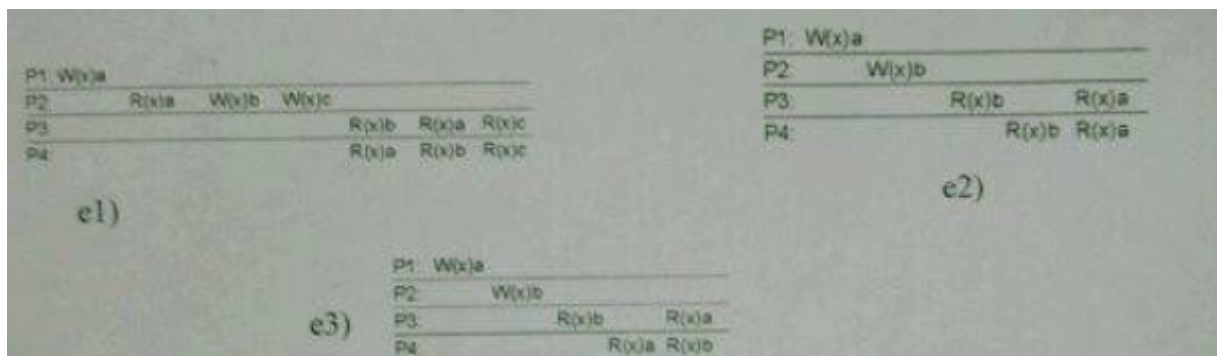
Koji rezultati print naredbi nisu moguci pod uslovom da je skladiste podataka sekvencijalno konzistentno? Dati ukratko obrazlozenje.

x y z
 0 0 0
 0 0 1
 0 1 0
 0 1 1
 1 0 0
 1 0 1
 1 1 0
 1 1 1

0 1 0 Nije moguće jer print y mora da ide pre print x sto znači da bi i y bilo 0
 0 1 1 Nije moguće jer print y mora da ide pre print x sto znači da bi i y bilo 0

61. Definisati:

- striktnu
 Najjaci model konzistentcije. Bilo koja operacija nad podatkom x vraca rezultat poslednje write operacije nad podatkom x.
- sekvencijalnu
 Slabiji model od striktnog, najjaci model koji se moze ostvariti u DS.
 Rezultat bilo kog izvršenja je isti kao da su operacije svih procesa na skladištu podataka izvršene u nekom sekvencijalnom redosledu i operacije svakog pojedinacnog procesa pojavljuju se u redosledu koji je odredjen njihovim programom.
- uslovnu
 Slabiji model od sekvencijalne. Upisi koji su potencijalno uslovljeni moraju se videti u svim procesima u istom redosledu. Konkurentni upisi se mogu videti u razlicitim procesima u razlicitom redosledu.
- FIFO konzistenciju
 Upisi koje obavi jedan proces se vide od strane drugih procesa po redosledu u kome su izdati. Ali upisi razlicitih procesa mogu se videti u razlicitom redosledu u razlicitim procesima.
- Koja od skladišta podataka sa slike su sekvencijalno konzistentna a koja su uslovno konzistentna a koja FIFO konzistentna?
Sekvencijalno: e2
Kauzalno: e1, e2, e3
FIFO: e1, e2, e3



62. Koji od događaja prikazanih na slici mogu da nastupše u slučaju

- Kauzalno konzistentnog skladišta podataka **F, E, D, C, B**
- Sekvencijalno konzistentnog skladišta podataka **F, D, B**
- Striktno konzistentnog skladišta podataka **F**

A.	P1	W(x)a			
	P2		R(x)a	W(x)b	
	P3				R(x)b
	P4			R(x)a	R(x)b

B.	P1	W(x)a			
	P2		R(x)a	W(x)b	
	P3				R(x)a
	P4				R(x)b

C.	P1	W(x)a			
	P2		W(x)b		
	P3			R(x)b	R(x)a
	P4			R(x)a	R(x)b

D.	P1	W(x)a			
	P2		W(x)b		
	P3			R(x)a	R(x)b
	P4			R(x)a	R(x)b

E.	P1	W(x)a			
	P2		W(x)b		
	P3			R(x)a	R(x)b
	P4			R(x)a	R(x)a

F.	P1	W(x)a			
	P2		W(x)b		
	P3			R(x)b	R(x)b
	P4			R(x)b	R(x)b

63. Koje vrste replika postoje? Koje se informacije mogu proslediti replikama kada se obavlja azuriranje?

Tri tipa replika

- Permanentne replike
- Replike inicirane od strane servera
- Replike inicirane od strane klijenta

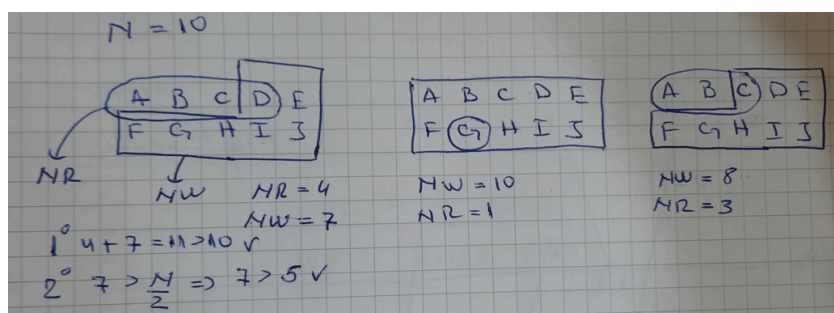
Informacije koje se prosledjuju replikama kada se obavlja azuriranje

- Samo obaveštenje o obavljenom ažuriranju
- Podaci koji su ažurirani
- Operacija koje su izazvale ažuriranje.

64. Fajl je repliciran na N servera, Ako se koristi Giffordov algoritam za postizanje konsenzusa navesti koji uslovi moraju biti zadovoljeni i sta koji uslov garantuje. Ako je $N = 10$ navesti tri dozvoljene kombinacije read i write kvoruma?

Moraju da vaze sledeci uslovi za NR i NW

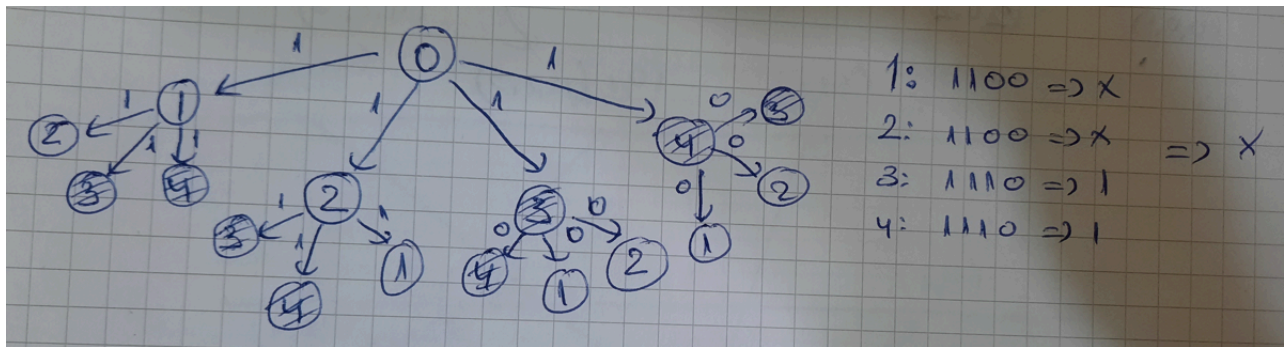
- $NR + NW > N$
 - Spreca read-write konflikte
- $NW > N / 2$
 - Spreca write-write konflikte



65. U distribuiranom sistemu postoji 5 procesa koji treba da se usaglase kod donosenja odluke. Ako dva procesa generisu pogresne rezultate da li je moguće doneti odluku? Ilustrovati promerom sa jednim generalom i 4 porucnika.

Mora da se ispostuje

- Svi lojalni porucnici donose istu odluku
- Ako je general lojalan svi lojalni postuju tu odluku

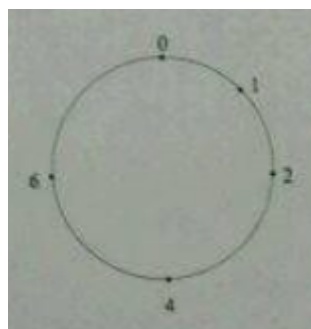


Potrebno je da za m procesa koji ne daju tačan rezultat imamo $2m + 1$ dobrih procesa.

$m = 2, n = 5$

$2m + 1 = 5 \Rightarrow$ **potrebno je 7 procesa da bi se donela odluka sa 2 neispravna procesa, odnosno 5 ispravnih, ukupno ucesnika je $3m + 1 = 7$.**

66. Na slici je prikazan Chord prsten koji koristi $n = 2^3$ identifikatora. Punim krugovima oznaceni su peer cvorovi i njihovi identifikatori. Prikazati kojim cvorovima ce biti dodeljeni objekti sa ključevima $k_1 = 1, k_2 = 3, k_3 = 5, k_4 = 7$.



1 = K_1 , 4 = K_2 , 6 = K_3 , 0 = K_4

67. Sta je funkcija middleware u distribuiranom sistemu?

Osnovni cilj middleware je da se sakrije heterogenost platforme na kojoj je sistem izgrađen od aplikacije.

68. Opisati sta se podrazumeva pod skalabilnoscu distribuiranog sistema

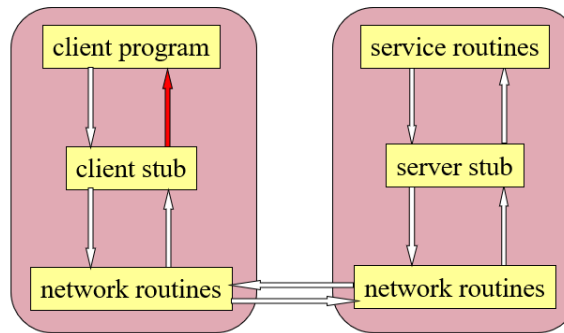
Podrazumeva se sistem koji ima mogucnost prosirenja tj. dodavanja novih racunara.

Skalabilnost se može posmatrati kroz tri dimenzije:

- Skalabilnost u odnosu na broj korisnika i resursa
- Skalabilnost u odnosu na geografsku udaljenost resursa i korisnika
- Administrativna skalabilnost – sistemom se može lako upravljati cak i ako se prostire kroz više administrativnih domena.

69. Pobrojati redom aktivnosti koje se izvrsavaju prilikom poziva udaljene procedure.

1. Klijent poziva lokalnu proceduru klijent stub.
2. Klijent stub pakuje argumente (marshaling) za udaljenu proceduru i gradi jednu ili vise poruka i poziva lokalni OS.
3. Lokalni OS salje poruku udaljenom OS-u (koriscenjem transportnog protokola TCP ili UDP)
4. Prijem poruke: mrezna poruka stize do odredista, OS prosledjuje poruku serverskom stub-u. Serverska stub funkcija prima poruku od lokalnog OS, raspakuje je (unmarshaling) i izvlaci argumente za poziv lokalne procedure.
5. Serverski stub poziva zeljenu serversku proceduru i predaje joj parametre koje je primio od klijenta.
6. Server izvrsava pozvanu proceduru i vraca rezultat stub-u.
7. Serverski stub pakuje rezultat u poruku i poziva svoj lokalni OS.
8. Slanje poruke kroz mrezu: Serverski OS salje poruku klijentskom OS-u
9. Lokalni OS prosledjuje poruku klijent stub-u
10. Klijent stub izvlaci rezultat iz poruke i vraca klijentskom procesu.



70. Izlaz iz IDL kompajlera sastoji se od tri fajla. Koji su to fajlovi i sta sadrze?

Poziv idl kompajlera: **idl primer.idl**

- **header fajla (npr. primer.h)**
 - Header fajl sadrži jedinstveni identifikator, definiciju tipova, konstanti i prototipova funkcija.
 - On treba da bude uključen (korišćenjem #include) i u klijent i u server kod
- **klijent stub (primer_cstub.c)**
 - Klijent stub sadrži procedure koje će klijent program pozivati
 - Ove procedure su zadužene za pakovanje parametara u poruke, slanje poruka, prijem poruka, izvalačenje rezultata poziva procedure i prosleđivanje klijentu
- **server stub (primer_sstub.c)**
 - Sadrži procedure koje se pozivaju kada stigne poruka do servera i koje zatim pozivaju odgovarajuću serversku proceduru

71. Simbol $\rightarrow$ označava Lamportovu relaciju “desilo se pre” a i b su događaji, V_a je vektorski casovnik dužine n koji predstavlja vreme kada je događaj a nastupio.

- a. Koja karakteristika sistema određuje n? **broj procesa**
- b. Definirati šta znači $V_a = V_b$ i $V_a > V_b$ **svi elementi vektorskog casovnika V_a su kao i elementi vektorskog casovnika V_b , i to važi za svaki element $V[i]$, $i = 1, n$ $V_a > V_b$, ista stvar samo su veći**
- c. Ako je $a \rightarrow b$ šta znamo o V_a i V_b ? **$V_a < V_b$**
- d. ako ne važi $a \rightarrow b$ niti je $b \rightarrow a$ šta znamo o V_a i V_b ? **pa ništa, događaji a i b su konkurentni**

72. Kako je na jedinstven način indentifikovana udaljena procedura kod Sun RPC?

Ilustrovati na koji način je to urađeno u klijentskoj aplikaciji. - 02.2. DS-P 23:10

Objasniti ulogu portmappera kod SunRPC poziva udaljene procedure? **-(ima vec)**

73. Definicija interfejsa kod DCE RPC. - 47 slajd, 2.prez

Koja semantika poziva udaljenih procedura je podržavana kod DCE RPC i kako? - 50 slajd. , 2.prez

Povezivanje klijenta i servera kod DCE RPC? - 53 - 56 slajd, 2.prez

74. a. Podela distribuiranih sistema u odnosu na oblast primene?

b. Šta predstavlja transakcija u distribuiranim informacionim sistemima? Koje osobine treba da zadovolji svaka transakcija da bi bila uspešno izvršena? - **ima vec 21. pitanje**

75. a. Šta predstavlja i koja je uloga reference udaljenog objekta u distribuiranom sistemu?

- 59 slajd, 2.prez (prve dve crne recenice)

b. Povezivanje klijenta i servera kod poziva metoda udaljenog objekta - 60 slajd, 2.prez